Review on Maximum Weighted Internal Spanning Tree Problem

Jahid Hasan (ID: 1024052109)
Course: CSE 6410 Group: 8

August 21, 2026

Abstract

The Maximum Weighted Internal Spanning Tree (MaxWIST) problem is a well-known NP-hard problem in Graph Theory. Given a vertex-weighted connected graph $G = (V, E)$ with n vertices, the goal is to find a spanning tree T that maximizes the sum of its internal vertices. This problem has significant applications in the real world such as network design, circuit layout, database systems, and social network analysis. This review paper is focused on recent advances in approximation algorithms for the MaxWIST problem, especially for d-regular graphs. We summarize the key study that provides a novel approximation algorithm for d-regular graphs with a tighter upper bound, and we also summarize the results on different types of graph subdivisions.

1 Introduction

Due to the numerous applications and complexity, the Maximal Internal Spanning Tree (MaxIST) problem and its weighted variant (MaxWIST) holds significant importance in graph theory. The MaxWIST generalizes the Hamiltonian Path problem. As compare to other classical problem related with Spanning tree such as Minimum Spanning tree has polynomial time solutions using Kruskal's Algorithm or Prim's Algorithm we can solve this in $O(|E| \log |V|)$, To solve Maximal Internal Spanning tree related problem research focusing on developing efficient approximation algorithm specially for restricted structures graph those are bounded by vertex degree.

d-regular graph is also a restricted structures graph where every vertex will have exactly d degree. Cubic graphs ($d = 3$) is a type of d-regular graph which have been studied extensively but for $d \geq 4$ are less common in this field. This review examines a comprehensive study by Hakim et al. [1] that makes a significant contribution to $d \geq 4$ regular graphs by providing a generalized approximation algorithm. It established a tight upper bound of broader graph classes via its subdivisions.

2 Motivation

The paper [1] provides a novel technique to solve a NP-hard problem for a special type of graphs. It brings a technique to study traditional spanning tree optimization with modern vertex-centric graph analysis. It uses approximation theory which shows how we can solve NP-hard optimization problems in graph theory. From this algorithmic innovation, we can develop new frameworks applicable to broader classes of graph optimization problems.

- In a network system we can maximize the capacity at hub routers while ensuring all nodes remain connected.
- In VLSI Design we can minimize leaf components while maximizing internal processing units.
- In a database system design we can optimize query processing trees by emphasizing internal join operations.
- In social network analysis we can identify influential users who bridge different communities.

3 Problem Definition

Given a vertex-weighted graph $G = (V, E)$ with n vertices, we have to find a spanning tree T of the given graph G that maximizes the sum of the weights of internal vertices:

$$w(I(T)) = \sum_{v \in I(T)} w(v),$$

where $I(T)$ is the set of internal vertices of T (i.e., vertices of degree at least 2 in T).

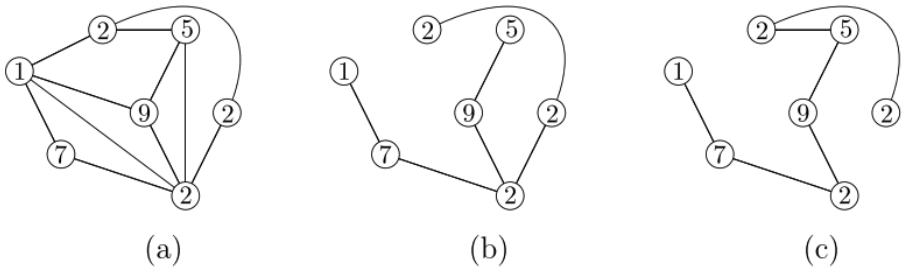

Figure 1: (a) A vertex-weighted graph G.
(b) A spanning tree of graph G.
(c) A maximum weighted internal spanning tree of graph G.

4 Historical Context and Prior Work

In 2008, a research paper [2] titled "On finding spanning trees with few leaves" started study on MaxIST for cubic ($d = 3$) graphs. This paper shows a 5/6 approximation for MaxIST using LP relaxation. This was the first impactful research in this field, though the graph is restricted to degree 3.

In 2018, Ahmad Biniaz published a research paper [3] where, for the first time, he started studying this problem for a weighted version. He gave a 3/4 approximation for MaxWIST in cubic ($d = 3$) graphs. He first bridged the unweighted to weighted case but still focused on degree 3 graphs.

Until Hakim et al. published a research paper [1], the problem was thought to be NP-hard for d-regular graphs where $d \geq 4$. In 2024, they generalized the study and gave an $O(dn)$ algorithm for d-regular graphs. Their approximation algorithm achieves

$$\frac{\beta_d}{\beta_d + d - 2}, \quad \text{where } \beta_d = (d-1)H_{d-1},$$

which is the same for [2] and [3] and is better for $d \geq 4$ graphs.

5 Regular Graphs and Their Special Properties

Before driving into the approximation algorithm for MaxWIST, let's understand why regular graphs are special in graph theory. We already know that a d-regular graph is a regular graph where each vertex has exactly d degree. It means that each of the vertices will have exactly same structure by which we can define following constraints:

- The total number of edges: $|E| = \frac{nd}{2}$
- It's guaranteed that $\alpha(G) \geq \frac{n}{d+1}$, giving us a nice lower bound on the independence number, where $\alpha(G)$ is the size of maximum independent set in the unweighted case.
- Any independent set S satisfies some interesting size constraints that don't hold in general graphs.

From the above constraint, even if we apply greedy algorithm on such graph, the greedy algorithm doesn't need to worry about degree-based tie-breaking.

6 Algorithm and Techniques

Hakim et al.[1] has contributed a significant study on MaxWIST and they provide a Greedy DFS algorithm that can solve the problem described in the above section along with its subdivisions graph in polynomial time. They provided an efficient unified approximation approach.

6.1 Greedy DFS for d-Regular Graphs

The core idea of this Greedy DFS approach is to construct a spanning tree T by making locally optimal choices during its depth-first traversal.

1. **Root Selection:** First, it selects a root vertex r with a minimum total weight in its closed to neighborhood $N(r)$. This initial step aims to minimize the "waste" of having a high-weight vertex as a leaf due to its placement early in the tree.

2. **Greedy Traversal:** Starting from the root node r, the algorithm performs a depth-first search (DFS). At each step, from the current vertex, it chooses the next vertex x to visit that maximizes the ratio
$$\frac{w(x)}{u(x)},$$
where $w(x)$ is the vertex weight and $u(x)$ is its number of unvisited neighbors.

6.1.1 Construction of MaxWIST (Maximal Backward Paths)

The analysis of the algorithm's performance relies on a careful examination of the structure of the resulting DFS tree T. The key concept is the maximal backward path originating from a leaf. A backward edge connects a vertex to an ancestor in T. For a leaf vertex x which has degree 1 in T, the existence of $d-1$ backward edges is guaranteed. The analysis of the algorithm constructs paths by following these backward edges and their subsequent tree edges.

Lemma 3.1 from Hakim et al.[1], establishes that the total weight of internal vertices along such a path originating from a leaf x is proportional to $w(x)$. By summing these inequalities for all leaves and considering that each internal vertex can be an endpoint for at most $d-2$ such paths (and the root for $d-1$), the following key inequality is derived:

$$(d-2)w(I) \geq \beta_d\big(w(V) - w(I)\big),$$

where

$$\beta_d = (d-1)H_{d-1}$$

and H_{d-1} is the $(d-1)$-th harmonic number. Rearranging this inequality yields the approximation ratio:

$$\frac{w(I)}{w(V)} \geq \frac{\beta_d}{\beta_d + d - 2}.$$

For $d = 3$, this gives $\beta_3 = 3$, recovering the known 3/4-approximation. For larger d, β_d grows, leading to an improved ratio compared to previous unweighted bounds.

6.2 Extension to Subdivisions of d-Regular Graphs

A major advantage of the greedy DFS techniques is its adaptability to more graph classes. Hakim et al.[1], demonstrate its application to other subdivisions of d-regular graphs, where each edge is replaced by a path of degree-2 vertices.

The same greedy DFS algorithm is applied to the subdivision graph. The significant difference arises when a leaf node x in the DFS tree is a subdivision vertex. In this case, x has only one backward edge. The analysis shows that even with this single backward chain, the weight accumulation along the path is sufficient.

For a subdivision of a d-regular graph, the greedy DFS algorithm achieves an approximation ratio of:

$$\frac{w(I)}{w(V)} \geq \frac{d+3}{d-1} - \epsilon$$

This result is particularly worthy because it shows that the algorithm remains well performed even if the graph contains long paths of degree-2 vertices, which is a structure that is typically being challenged for spanning tree optimization.

7 Performance Analysis

Here in this section we will try to provide a comparative study of the performance of approximation algorithms for the MaxWIST problem. We will focus on approximation ratio, time complexity, and experimental validation.

7.1 Approximation Ratios

The performance of the approximation algorithm is primarily measured by its ratio, which guarantees that the solution which is found is with a certain factor of the optimal solution.

7.1.1 Upper Bound Approximation Ratio

Upper Bound Approximation ratio is the limit that tells how well any polynomial-time algorithm can perform unless it is P=NP. This paper mentioned an algorithm which has constructed infinite families of d-regular graphs to show that for any spanning tree, the ratio $w(I)/w(V)$ cannot be larger than $d/(d+1)$.

7.1.2 Lower Bound Approximation Ratio

Lower Bound Approximation Ratio is the ratio which is guaranteed by a specific algorithm. Hakim et al.[1]'s research has achieved a lower bound of $\beta_d/(\beta_d + d - 2)$.

7.2 Time Complexity

The time complexity is the computational efficiency of an algorithm.

For d-regular graphs: The greedy DFS algorithm runs in $O(dn)$ time, where n is the number of vertices and d is the degree. This is linear in the graph size for fixed d.

Table 1: Approximation Ratios for MaxWIST in d-Regular Graphs

Graph Class	Upper Bound	Lower Bound	Researchers	Year
Cubic ($d = 3$)	$3/4 = 0.75$	$3/4 = 0.75$	Biniaz[3],	2021
4-regular	$4/5 = 0.80$	≈ 0.733	Hakim et al.[1],	2024
5-regular	$5/6 \approx 0.833$	≈ 0.735	Hakim et al.[1],	2024
General d-reg.	$\frac{d}{d+1}$	$\frac{\beta_d}{\beta_d+d-2}$	Hakim et al.[1],	2024

For Subdivisions graphs: The algorithms for x-subdivisions graphs of Hamiltonian graphs run in $O(n)$ time, as they rely on finding a cycle and processing chains.

Comparison: This is a significant improvement over some earlier approximation algorithms for related problems, which had higher polynomial time complexities (e.g., $O(n^4)$).

7.3 Comparative Analysis of Algorithms

The following table will give a solid comparison of the discussed algorithms, by highlighting the trade-offs between graph generality, approximation quality, and efficiency.

Table 2: Algorithm Comparison for MaxIST/MaxWIST Problems

Algorithm / Study	Graph Class	Problem	Approx. Ratio	Time Complexity	Key Contribution
Salamon (2008) [2]	Cubic ($d = 3$)	MaxIST	$5/6 \approx 0.833$	Polynomial	First major approximation for cubic graphs
Biniaz (2021)[3]	Cubic ($d = 3$)	MaxWIST	$3/4 = 0.75$	$O(n)$	First dedicated algorithm for weighted case
Hakim et al. (2024)[1]	d-regular ($d \geq 3$)	MaxWIST	$\frac{\beta_d}{\beta_d+d-2}$	$O(dn)$	Generalization to d-regular graphs
Hakim et al. (2024)[1]	Subdivisions of Hamiltonian Graphs	MaxWIST	$1 - \frac{2}{x-1}$	$O(n)$	Linear-time algorithm for broad class

8 Open Challenges

8.1 General Graphs

So far, the polynomial-time solutions have been found mainly for d-regular graphs. For general graph classes, finding a maximum weighted independent set of a tree (MaxWIST) remains an NP-hard problem.

8.2 Tighter Approximation Ratios

An open question is whether one can design an algorithm that achieves an approximation ratio closer to $\frac{d}{d+1}$, or whether the hardness bounds can be improved further for weighted independent sets on trees.

8.3 Complexity for Other Subdivision Graphs

Hakim et al.[1]'s research paper studies subdivisions of planar graphs; however, the complexity for general planar graphs remains open. Specifically, it is still unknown whether MaxWIST is NP-hard on planar graphs. Similarly, the computational complexity for bipartite graphs has not been well established.

8.4 Complexity of Budgeted MaxWIST

In the *Budgeted MaxWIST* problem, one seeks a spanning tree that maximizes the weight of internal vertices while ensuring the total sum of vertex weights does not exceed a given budget. The complexity of this variant, even on regular graphs, remains an open question.

9 Future Works

While greedy DFS approach by Hakim et al.[1] got a success in this problem, there is a huge chance to develop more powerful techniques.

9.1 Opportunities for New Algorithms and Heuristics

The success of LP-based algorithms for the unweighted MaxIST problem, That suggests the similar approaches could be applied to MaxWIST. Designing a strong LP relaxations with effective rounding schemes could potentially help to approximate the ratios that can close the gap with the known upper bounds for d-regular graphs.

9.2 Applications in Networks, Distributed Systems, and Optimization

The MaxWIST problem is not just aligned in theory, It is also perfectly aligned with real world problems such as Networks, VLSI design, Distributed System etc. Future work should focus on bridging the gap between theory and applications.

9.2.1 Network System

In communication networks an internal vertex can represent a router. A MaxWIST solution could define a backbone network that maximizes the total capacity of these routing nodes. Research could study MaxWIST under these network-specific constraints like bandwidth, latency, and node failure models.

9.2.2 Distributed System

In a Distributed System for massive graphs that cannot be loaded into a memory, designing distributed algorithms for MaxWIST is a challenging task. How can nodes, communicating only with neighbors, collaboratively build a spanning tree that approximates the maximum

weight of internal objective. Similarly, developing streaming algorithms that make a single pass over the edge list could be so valuable for processing large-scale graph data.

9.2.3 Database System

In distributed databases, the query execution plan can be modeled as a tree. Internal nodes represent operations like joins that require significant computational resources. A MaxWIST like approach could be adapted to optimize the placement of these expensive operations on the most powerful servers, so that it minimizes the total query time.

9.3 Integration with Machine Learning

Many Machine Learning models like GNN (Graph Neural Network) use Graph algorithms to build a model with good performance. The performance of approximation algorithms often depends on hidden graph properties. An ML model could analyze a given graph's features and automatically select the best algorithm of MaxWIST solvers. Also using Machine learning, we can guide the heuristic choice.

10 Conclusion

The MaxWIST problem still remains a challenging task in the area of research on graph optimization. The work done by Hakim et al.[1] provides a significant advance for d-regular graphs by establishing a tight upper bound and a generalized approximation algorithm, which improves the results for $d \geq 4$. Their extension of the problem to graph subdivisions also opens up new research areas.

The Key open challenges still exist, including closing the gap between the current approximation ratios and the hardness bounds, as well as understanding the complexity of the weighted versus the unweighted versions of this problem.

References

[1] Hakim, S. A., Nishat, R. I., & Rahman, M. S. (2024). *Approximation algorithms for maximum weighted internal spanning trees in regular graphs and subdivisions of graphs.* The Computer Journal, 67(12), 2898–2905.

[2] Salamon, G., & Wiener, G. (2008). *On finding spanning trees with few leaves.* Information Processing Letters, 105(4), 164–169.

[3] Biniaz, A. (2021). *Better approximation algorithms for maximum weight internal spanning trees in cubic graphs and claw-free graphs.* In International Workshop on Algorithms and Computation (pp. 260-271). Springer, Cham.

[4] Salamon, G. (2009). *Approximating the maximum internal spanning tree problem.* Theoretical Computer Science, 410(52), 5273–5284.